RetinaScan-XAI -- Reviewed Implementation Blueprint

SIH 2026 · PS38 · MathWorks Sponsored

3-Person Team · 4-Week Build · Zero-Block Parallel Workflow

PART A -- Architectural Review & Critique

The following issues were identified in the existing codebase and prior plan. Every fix listed here is incorporated into the rewritten blueprint in Part B.

CRITICAL: Preprocessing Train/Inference Mismatch

Problem: train_model.py trains with only standard ImageNet normalization (transforms.Normalize([0.485...). But inference.py now applies CLAHE + Ben Graham normalization before the same ImageNet normalize. The model has never seen CLAHE/Ben-Graham enhanced images during training.

Impact: The model's learned feature distributions do not match the inference input distribution. This silently degrades accuracy on every live upload. The 97.6% sensitivity number was measured on raw images -- it does not hold on enhanced images.

Fix (Week 1, P1): Add CLAHE + Ben Graham as the first transform in train_model.py's training and validation pipelines, then retrain the model. Alternatively, remove CLAHE from inference.py to match training. The pipeline must be identical end-to-end.

CRITICAL: Data Leakage Risk in Evaluation

Problem: train_model.py and evaluate_model.py both use train_test_split(df, test_size=0.20, random_state=42, stratify=...). They share the same CSV and the same seed, so the val set should be identical. However, there is no explicit mechanism to guarantee this. If the CSV is re-sorted, or a row is added/removed between runs, the splits silently diverge and evaluation metrics become meaningless.

Impact: A single row change in train.csv means evaluate_model.py is testing on images the model was trained on, inflating sensitivity/specificity numbers.

Fix (Week 1, P1): Split once, save train_ids.csv and val_ids.csv to disk. Both scripts load from these frozen split files. Never re-split.

HIGH: Training Uses 0.50 Threshold, Evaluation Uses Youden's J

Problem: train_model.py line 196 hardcodes binary_pred = (referable_prob >= 0.5) for its per-epoch sensitivity logging. But evaluate_model.py uses Youden's J to find 0.1162. The numbers printed during training are misleading -- they suggest lower sensitivity than the model actually achieves.

Fix: Use the same threshold logic everywhere, or clearly label the training logs as "at default 0.50 threshold" so nobody confuses them with the final tuned metrics.

HIGH: OOD Detector is Fragile

Problem: validate_fundus() uses two hardcoded heuristics: corner brightness > 0.35 and red-channel dominance. Many legitimate fundus cameras (especially wide-field or smartphone adapters) produce images with bright corners. Some healthy fundus images have green-tinted illumination.

Impact: False rejections of real patient scans during a live demo would be catastrophic.

Fix (Week 3, P2): Add a confidence-based OOD fallback: if the model's top-class softmax probability is below 0.40, flag it as "Low

Confidence -- Possible Non-Fundus Image" rather than hard-rejecting. Keep the heuristic checks as warnings, not blockers.

MEDIUM: Model Architecture Defined in 3 Different Files

Problem: build_model() is copy-pasted identically in train_model.py, evaluate_model.py, messidor_test.py, and inference.py. If anyone changes the fc layer structure in one file but not the others, the .pth weights will fail to load with a cryptic state_dict key mismatch error.

Fix (Week 1, P1): Extract build_model() into a single shared file: ml_script/model.py. All scripts import from it.

MEDIUM: No CLAHE During Evaluation

Problem: evaluate_model.py uses plain val_transforms (resize + normalize). If training is updated to include CLAHE (per the fix above), evaluation must also include it. Currently it does not.

Fix: evaluate_model.py must import and apply the exact same preprocessing chain used in training.

LOW: Grad-CAM Hook Leak

Problem: GradCAMSimple.__init__ calls register_forward_hook() and register_full_backward_hook() but never removes them. On repeated calls to run_inference(), hooks accumulate on the singleton model, causing memory leaks and increasingly slow inference.

Fix (Week 2, P2): Store hook handles and call .remove() after each Grad-CAM generation, or instantiate GradCAMSimple once alongside the singleton model.

LOW: `messidor_test.py` Cannot Compute Real Accuracy

Problem: The MESSIDOR-2 images in the dataset folder have no per-image grade labels. The script compares predicted prevalence against a published aggregate number (46.5%). This is not a real validation -- it cannot compute sensitivity, specificity, or AUC.

Fix: Document this explicitly. Label this test as "Distribution Plausibility Check" -- not "External Validation."

PART B -- Rewritten Implementation Blueprint

Team Roles

ID	Role	Owns	Core Skills
P1	ML Engineer	`ml_script/`	Python, PyTorch, OpenCV, scikit-learn
P2	Backend Engineer	`Backend/`	Python, FastAPI, REST APIs, OpenCV
P3	Frontend Engineer	`Frontend/retinascan-app/`	React, CSS, Vite, JavaScript

Repository File Structure

Every file is tagged with its owner ([P1], [P2], [P3]) and a description of exactly what it does.

```
RetinascanXAI/
|
|-- README.md                          [All]  Project overview, setup guide, API reference
|-- .gitignore                         [All]  Ignores .pth, node_modules, DataSet/, __pycache__/
|-- model_link.txt                     [P1]   Google Drive URL to download the trained .pth file
|
```

```
|-- install_deps.bat                    [P2]   Installs all Python + Node dependencies in one click
|-- start_backend.bat                   [P2]   Starts FastAPI on localhost:8000
|-- start_frontend.bat                  [P3]   Starts Vite dev server on localhost:5173
|
|-- ml_script/                          <- P1 OWNS THIS ENTIRE FOLDER
|   |-- model.py                        [P1]   SINGLE source of truth for build_model()
|   |-- preprocess.py                   [P1]   SINGLE source of truth for CLAHE + Ben Graham + transforms
|   |-- split_dataset.py                [P1]   Runs ONCE to create train_ids.csv + val_ids.csv
|   |-- train_ids.csv                   [P1]   Frozen training image IDs (output of split_dataset.py)
|   |-- val_ids.csv                     [P1]   Frozen validation image IDs (output of split_dataset.py)
|   |-- train_model.py                  [P1]   ResNet-50 training loop using frozen splits + CLAHE
|   |-- evaluate_model.py               [P1]   Runs on val set, outputs sensitivity/specificity/AUC
|   |-- messidor_test.py                [P1]   Distribution plausibility check on MESSIDOR-2
|   |-- visualise_metrics.py            [P1]   Generates ROC curve, confusion matrix PNGs
|   |-- evaluation_report.json          [P1]   Final locked metrics JSON
|   |-- matlab/
|       |-- segment_lesions.m           [P1]   MATLAB lesion segmentation (MathWorks Toolbox)
|       |-- README.md                   [P1]   Instructions for running MATLAB scripts
|
|-- Backend/                            <- P2 OWNS THIS ENTIRE FOLDER
|   |-- main.py                         [P2]   FastAPI server: /upload, /health, /eval-report
|   |-- inference.py                    [P2]   Model load + CLAHE + Grad-CAM + OOD check
|   |-- requirements.txt                [P2]   All Python dependencies
|   |-- retinascan_resnet50.pth         [P1->P2] Trained weights (gitignored, from Drive)
|
|-- Frontend/retinascan-app/            <- P3 OWNS THIS ENTIRE FOLDER
    |-- index.html                      [P3]   HTML entry point
    |-- vite.config.js                  [P3]   Vite build config
    |-- package.json                    [P3]   Node dependencies
    |-- public/images/                  [P3]   Static demo fundus images + heatmaps
    |-- src/
        |-- App.jsx                     [P3]   Navigation state machine + API fetch
        |-- data.js                     [P3]   Mock patient data (schema is FROZEN)
        |-- index.css                   [P3]   Global design tokens
        |-- components/
            |-- Home.jsx                [P3]   Landing screen with 3 module cards
            |-- Upload.jsx              [P3]   2-column clinical intake form
            |-- QualityCheck.jsx        [P3]   Animated IQA gate (3 progress bars)
            |-- Processing.jsx          [P3]   Inference animation (conic ring + checklist)
            |-- Results.jsx             [P3]   Clinical results viewer
            |-- Canvas.jsx              [P3]   PACS workstation (lesion pins + Grad-CAM)
            |-- Queue.jsx               [P3]   Patient review queue
            |-- Referral.jsx            [P3]   Auto-generated referral letter
            |-- Admin.jsx               [P3]   Analytics dashboard
```

API Integration Contract

P2 and P3 agree on this on Day 1. Never break it.

`POST /upload` (FormData with `file` field)

Success: { "severity": 2, "confidence": 96.4, "label": "Moderate NPDR", "action": "Specialist Triage Required", "heatmap_url": "/images/heatmap_live_abc123.jpg", "all_probs": [0.01, 0.05, 0.72, 0.14, 0.08], "mode": "real" }

OOD Rejection: { "mode": "error", "error": "Retinal pigment signature not found." }

Mock Fallback: { "severity": 2, "confidence": 96.4, "label": "Moderate NPDR", "heatmap_url": "/images/heatmap_2_od.jpg", "mode": "mock" }

`GET /health` -> `{ "status": "ok", "model_loaded": true }`

`GET /eval-report` -> Returns `evaluation_report.json` contents

WEEK 1 -- Foundation & Architecture Fixes (All Independent)

P1 -- ML Engineer

Day	Task	File	Done When
1	Extract `build_model()` into shared mod	`ml_script/model.py`	All scripts import from here
1	Extract CLAHE + Ben Graham + transfo	`ml_script/preprocess.py`	Single preprocessing source of truth
2	Write dataset splitter, save frozen ID lis	`ml_script/split_dataset.py`	`train_ids.csv` + `val_ids.csv` exist
3-5	Update `train_model.py`: frozen splits +	`ml_script/train_model.py`	Trains 1 epoch without crashing
5	Start full 20-epoch training run	`.pth` file	Training running overnight

P2 -- Backend Engineer

Day	Task	File	Done When
1	Set up FastAPI with CORS + static file s	`Backend/main.py`	`curl localhost:8000/health` works
2	Implement `/upload` in MOCK mode	`Backend/main.py`	Returns hardcoded JSON
3	Write `requirements.txt`	`Backend/requirements.txt`	`pip install -r` succeeds
4	Write `.bat` scripts	`start_backend.bat`, `install_deps.bat`	Double-click works
5	Test frontend can fetch from `/upload`	--	No CORS errors

P3 -- Frontend Engineer

Day	Task	File	Done When
1	Initialize Vite + React, set up design tok	`index.css`, `package.json`	`npm run dev` works
2	Build `data.js` with mock patients	`data.js`	Exports all mock data
3	Build `Home.jsx` landing screen	`Home.jsx`	Renders in browser
4	Build `Upload.jsx` intake form	`Upload.jsx`	Demo cards auto-fill form
5	Wire navigation in `App.jsx`	`App.jsx`	Screen transitions work

WEEK 2 -- Core Pipeline (Soft Dependency: P1 -> P2)

P1 -- ML Engineer

Day	Task	File	Done When
1-2	Monitor training, fix divergence	`train_model.py`	val_acc > 75%
3	Upload `.pth` to Drive, update link	`model_link.txt`	P2 can download
4	Write evaluation with Youden's J thresh	`evaluate_model.py`	`evaluation_report.json` with sens >= 90%
5	Write ROC + confusion matrix plotter	`visualise_metrics.py`	PNG charts saved

P2 -- Backend Engineer

Day	Task	File	Done When
1	Write `inference.py` (preprocessing + G	`Backend/inference.py`	`run_inference()` callable
2	Wire real inference into `/upload`	`Backend/main.py`	Returns real grade
3	Fix Grad-CAM hook leak	`Backend/inference.py`	No memory growth across 20 uploads
4	Add `/eval-report` endpoint	`Backend/main.py`	Returns JSON

5	Download `.pth`, test real inference	--	Real heatmap generated

P2 uses random weights for Days 1-4. Only Day 5 needs P1's `.pth`.

P3 -- Frontend Engineer

Day	Task	File	Done When
1	Build `QualityCheck.jsx` (3 animated ba	`QualityCheck.jsx`	Animation plays, auto-advances
2	Build `Processing.jsx` (conic ring + che	`Processing.jsx`	Ring spins, steps resolve
3	Build `Results.jsx` (severity + probabilit	`Results.jsx`	Shows mock results
4	Wire full flow: Upload -> IQA -> Process	`App.jsx`	Demo flow runs
5	Polish animations	--	Smooth and demo-ready

WEEK 3 -- Advanced Features & Integration

P1 -- ML Engineer

Day	Task	File	Done When
1-2	Write MESSIDOR distribution check	`messidor_test.py`	`cross_dataset_report.json` generated
3	Run MATLAB `segment_lesions.m` (if a	`matlab/segment_lesions.m`	Lesion masks for 5 samples
4-5	Prepare presentation slides with metrics	Slides	All numbers ready for judges

P2 -- Backend Engineer

Day	Task	File	Done When
1	Soften OOD: low-confidence warning, n	`Backend/inference.py`	Dim fundus passes, watermelon rejected
2	Add `asyncio.Lock` inference guard	`Backend/main.py`	5 concurrent uploads don't crash
3	Verify static heatmap image serving	`Backend/main.py`	`<img src={heatmapUrl}>` works
4-5	**Integration with P3**: Test real API fro	--	Full connected flow works

P3 -- Frontend Engineer

Day	Task	File	Done When
1	Build `Canvas.jsx` (PACS workstation +	`Canvas.jsx`	Pins render with tooltips
2	Build `Queue.jsx` (review queue)	`Queue.jsx`	Patient list with severity badges
3	Wire `App.jsx` to `fetch()` P2's `/upload`	`App.jsx`	Real API populates Results
4	Handle OOD error response (alert + rec	`App.jsx`	Rejection shows message
5	Test full connected flow end-to-end	--	Real image -> real grade -> real heatmap

WEEK 4 -- Polish, Hardening & Demo Prep

P1 -- ML Engineer

Day	Task	Done When
1	Lock `evaluation_report.json` -- no more changes	File committed and frozen
2-3	Prepare judge Q&A answers	Can answer any model question instantly
4-5	Full demo rehearsal x 5	No crashes or unexpected results

P2 -- Backend Engineer

Day	Task	Done When
1	Stress test: 10 rapid uploads	Server survives burst load
2	Finalize `start_backend.bat` (UTF-8 encoding fix)	Double-click works
3-5	Demo rehearsal with P3	End-to-end works 5/5 times

P3 -- Frontend Engineer

Day	Task	File	Done When
1	Build `Referral.jsx`	`Referral.jsx`	Referral letter renders
2	Build `Admin.jsx`	`Admin.jsx`	Charts render
3	Add `Ctrl+Shift+R` demo reset	`App.jsx`	Instant reset works
4	Finalize `start_frontend.bat`	`start_frontend.bat`	Double-click works
5	Full 10-minute demo rehearsal	--	Flawless flow

Ground Rules

#	Rule	Why
1	**`data.js` schema is frozen**	P2's API keys must match P3's expected keys
2	**`model.py` is the ONLY place `build_model()` exists**	Prevents state_dict key mismatches
3	**`preprocess.py` is the ONLY place transforms are d**	Prevents train/inference mismatch
4	**Model weights on Google Drive, not Git**	`.pth` is 100MB+
5	**Commit per feature, not per day**	Branches: `ml/training`, `backend/inference`, `frontend/ui`
6	**P2 always returns valid JSON on error**	Frontend never sees Python traceback
7	**Demo threshold is `0.1162` -- locked after Week 2**	Produces 97.6% sensitivity
8	**`train_ids.csv` and `val_ids.csv` are NEVER regene**	Prevents data leakage